



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ

Rai Technology University

(Estd. under Karnataka Act. No. 49 of 2013)

ASSIGNMENT NO:02

Subject Title: Deep Learning

Subject Code:OCS351T

Submitted By:

Student Name: Mohammed Mubarak

University Enrolment Number:RTU24101CS08

Program Name:B.tech (CSE)

Semester: 5TH SEM

Session: 2026

Submitted To

Faculty Name: Mr.Satyam Sir

**Department of Computer Science &Engineering
College of Engineering**

Rai Technology University Bengaluru-561204

Introduction:-

Deep learning is a branch of Machine that uses neural networks to learn pattern from data. In the assignment the MNIST handwritten digit datasets is used to classify handwritten digits from 0 to 9 Tensorflow and Keras.

Objects:-

- * To load and explore the MNIST dataset
- * To display sample handwritten digit images
- * To design compile and train a Neural Network
- * To Evaluate the Model using test accuracy.
- * To visualize training and ~~validation~~ Validation accuracy and loss.

Dataset :-

The MNIST dataset contains handwritten digit images from 0 to 9. Each image is grayscale image of size 28×28 pixels. The data set is divided into 60,000 training images and 10,000 testing images.

Data preprocessing:-

The image pixel values are converted to float values and normalized by dividing by 255. This changes the pixel range from 0-255 to 0-1.

Neural Network Architecture:-

The baseline Model uses as input for 28×28 images a flatten layer, a Dense hidden layer with 128 Neurons ReLU activation and output Dense layer with 10 Neurons using softmax activation.

Model Compilation and Training:-

The Model was compiled using the Adam optimizer, Sparse categorical crossentropy loss function and accuracy as the evaluation metric. The Model was trained for 10 Epochs with a batch size of 32. twenty percent of the training data was used for Validation.

Model Evaluation:-

The trained baseline model was evaluated using the MNIST test dataset.

Testing of five Images:-

Image 1: Actual = — Predicted = —

Image 2: Actual = — Predicted = —

Image 3: Actual = — Predicted = —

Image 4: Actual = — Predicted = —

Image 5: Actual = — Predicted = —

Result:-

Baseline Test Accuracy = 97.73%.

Experiment:

For the Experiment the number of neurons in the hidden Dense layer was increased from 128 to 256

Experimental result:

Baseline Model: 128 Neurons - Test Accuracy = 97.73%

Experimental Model: 256 Neurons - Test Accuracy = 97.92%

Accuracy difference = 0.19 % points.

Comparison:

The Experimental model produced a test accuracy of 97.92% while the baseline model produced 97.73%

Conclusion:

The MNIST handwritten digit classification task was successfully implemented using Tensorflow and Keras.

The Neural Network was trained and Evaluated on the MNIST data set.

The baseline Model achieved 97.73% test accuracy. After increasing the hidden layer Neurons from 128 to 256 the Experimental Model achieved 97.92% test accuracy. This completed the required Experiment and Comparison.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

```
from tensorflow.keras import layers, models
```

```
print("TensorFlow Version:", tf.__version__)
```

```
TensorFlow version: 2.20.
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.
mnist.load_data()
```

```
print("Training images:", x_train.shape)
```

```
print("Training labels:", y_train.shape)
```

```
print("Testing images:", x_test.shape)
```

```
print("Testing labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/11490434/11490434 — 0s 0us/step
```

```
Training images: (60000, 28, 28)
```

```
Training labels: (60000,)
```

```
Testing images: (10000, 28, 28)
```

```
Testing labels: (10000,)
```

```
plt.figure(figsize=(10, 5))
```

```
plt.subplot(2, 5, i+1)
```

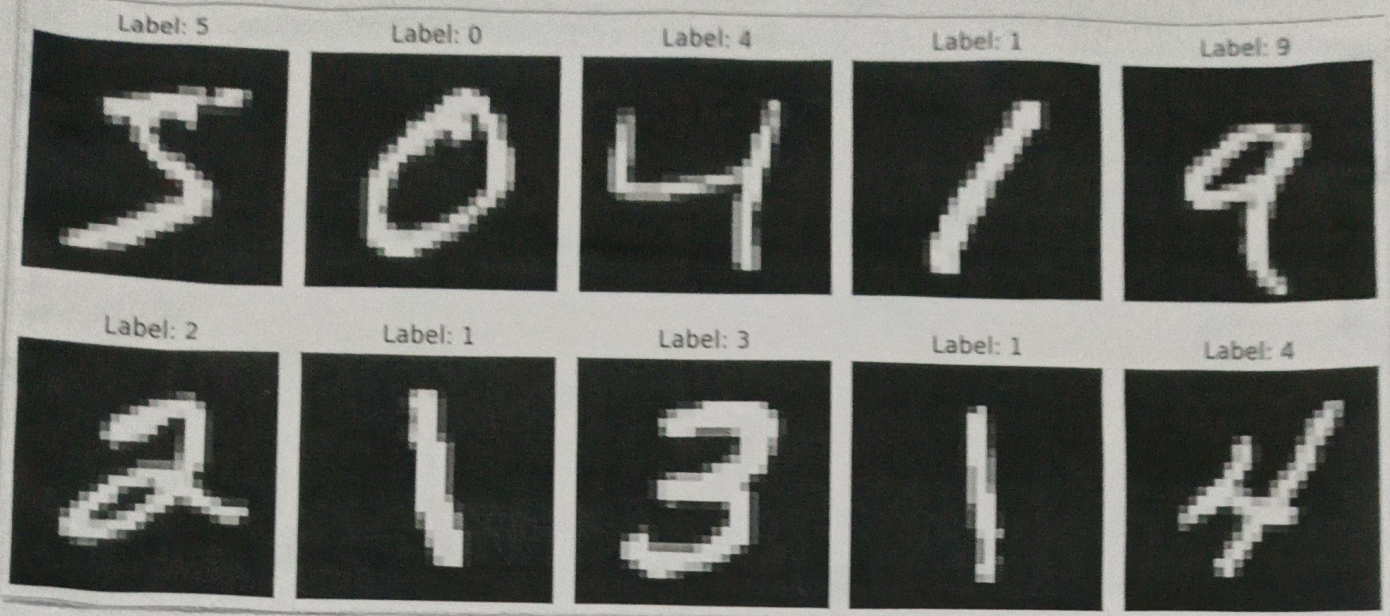
```
plt.imshow(x_train[i], cmap='gray')
```

```
plt.title(f"label: {y_train[i]}")
```

```
plt.axis('off')
```

plt.tight_layout()

plt.show()



```
x_train = x_train.astype("float32") / 255.0
```

```
x_test = x_test.astype("float32") / 255.0
```

```
print("preprocessing completed")
```

preprocessing completed

```
model = models.Sequential([  
    layers.Input(shape=(28, 28)),  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dense(10, activation='softmax')  
])
```

~~no~~

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

model.compile (

optimizer='adam'

loss='sparse_categorical_crossentropy',

metrics=['accuracy']

,

history = model.fit

x=train

y=train

epochs=10,

batch_size=32,

validation_split=0.2

```
Epoch 1/10
1500/1500 _____ 9s 6ms/step - accuracy: 0.9263 - loss: 0.2520 - val_accuracy: 0.9578 - val_loss: 0.1430
Epoch 2/10
1500/1500 _____ 7s 5ms/step - accuracy: 0.9697 - loss: 0.1031 - val_accuracy: 0.9658 - val_loss: 0.1131
Epoch 3/10
1500/1500 _____ 8s 5ms/step - accuracy: 0.9789 - loss: 0.0692 - val_accuracy: 0.9753 - val_loss: 0.0867
Epoch 4/10
1500/1500 _____ 10s 5ms/step - accuracy: 0.9859 - loss: 0.0479 - val_accuracy: 0.9753 - val_loss: 0.0844
Epoch 5/10
1500/1500 _____ 8s 5ms/step - accuracy: 0.9893 - loss: 0.0360 - val_accuracy: 0.9746 - val_loss: 0.0886
Epoch 6/10
1500/1500 _____ 8s 5ms/step - accuracy: 0.9923 - loss: 0.0261 - val_accuracy: 0.9756 - val_loss: 0.0858
Epoch 7/10
1500/1500 _____ 7s 4ms/step - accuracy: 0.9935 - loss: 0.0215 - val_accuracy: 0.9741 - val_loss: 0.0969
Epoch 8/10
1500/1500 _____ 9s 6ms/step - accuracy: 0.9948 - loss: 0.0170 - val_accuracy: 0.9769 - val_loss: 0.0870
Epoch 9/10
1500/1500 _____ 7s 5ms/step - accuracy: 0.9958 - loss: 0.0137 - val_accuracy: 0.9729 - val_loss: 0.1057
Epoch 10/10
1500/1500 _____ 8s 5ms/step - accuracy: 0.9969 - loss: 0.0106 - val_accuracy: 0.9758 - val_loss: 0.1037
```

```
test_loss, test_accuracy = model.evaluate(  
    x-test,  
    y-test,  
    verbose=0  
)
```

```
print("Test loss:", test_loss)
```

```
print("Test accuracy:", test_accuracy)
```

```
print("Test Accuracy: {test_accuracy * 100:.2f}%")
```

Test Loss: 0.08676881343126297

Test Accuracy: 0.9772999882698059

Test Accuracy: 97.73%

```
plt.figure(figsize=(12,15))
```

```
plt.subplot(1,2,1)
```

```
plt.plot(history.history['accuracy'], label='Training  
Accuracy')
```

```
plt.title('Training and Validation Accuracy')
```

```
plt.xlabel('epochs')
```

```
plt.ylabel('Accuracy')
```

```
plt.legend()
```

```
plt.subplot(1,2,2)
```

```
plt.plot(history.history['loss'], label='Training loss')
```

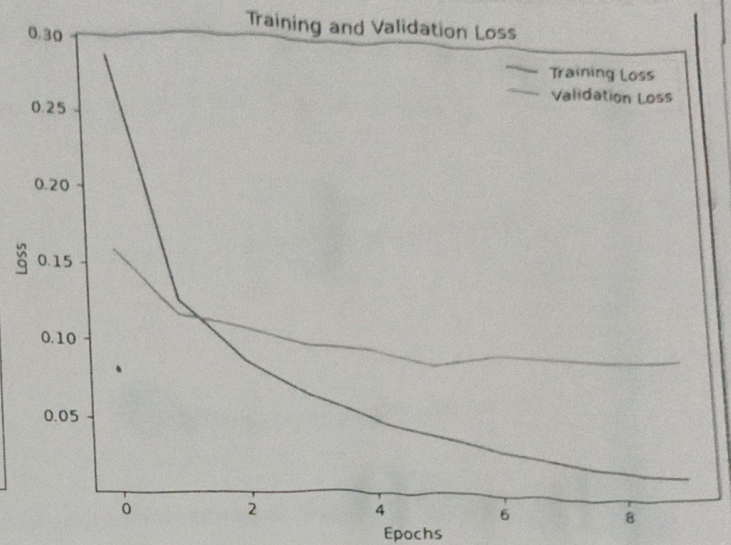
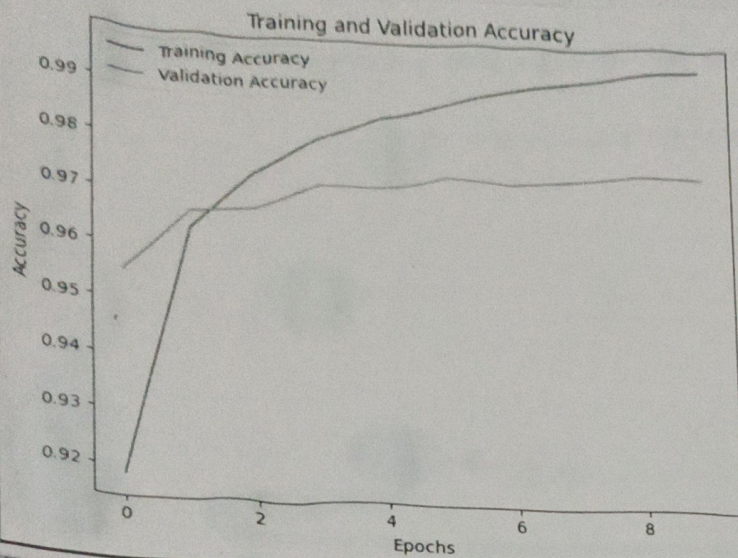
```
plt.plot(history.history['val_loss'], label='Validation  
loss')
```

```
plt.xlabel('epochs')
```

```
plt.ylabel('loss')
```

```
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

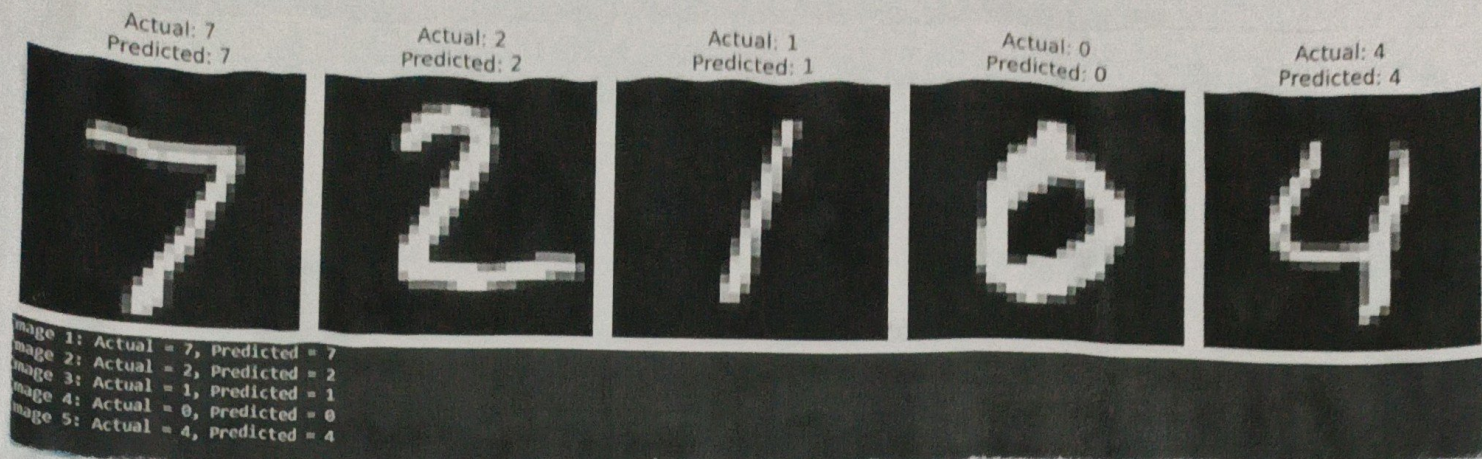


```
sample_images = x_test[:5]
actual_labels = y_test[:5]
predictions = model.predict(sample_images, verbose=0)
predicted_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(12, 4))

for i in range(5):
    plt.subplot(1, 5, i+1)
    plt.imshow(sample_images[i], cmap='gray')
    plt.title
        f"Actual: {actual_labels[i]} \n"
        f"Predicted: {predicted_labels[i]}"
    )
    plt.axis('off')

plt.tight_layout()
plt.show
```



Experiment - loss, Experiment - accuracy = Experiment - model
 .evaluate()

X-test,
 y-test,
 Verbose=0

)
 print("Experiment Test Accuracy:",
 f"{Experiment - accuracy * 100:.2f}%")

Experiment Test Accuracy 97.72%

print("Baseline Test Accuracy:",
 f"{Test - accuracy * 100:.2f}%")

print("Experiment Test Accuracy:",
 f"{Experiment - accuracy * 100:.2f}%")

difference = Experiment - accuracy - test accuracy

print("Accuracy Difference:",
 f"{difference * 100:.2f} percentage points")

Baseline Test Accuracy: 97.73%
Experiment Test Accuracy: 97.92%
Accuracy Difference: 0.19 percentage points